Architecture cible

Backend HBA Express — trajectoire d'evolution

Quatre paliers, chacun ouvert par un signal mesure — pas par une date

Ce document ne propose pas une refonte. Il decrit la trajectoire la moins couteuse pour que le backend actuel encaisse la croissance, en repoussant chaque decision structurante jusqu'au moment ou une mesure la rend necessaire. Chaque palier est donc introduit par son **signal declencheur**, jamais par une echeance.

Il s'appuie sur la structure relevee dans *ARCHITECTURE_BACKEND.pdf*, et sur une contrainte que le code documente lui-meme : **l'API ne peut pas etre repliquee aujourd'hui**. Tout le reste en decoule.

Palier	Objet	Signal declencheur
0	Lever le verrou de l'outbox	Immediat — il conditionne tous les autres
1	Tenir la charge sans rien extraire	CPU soutenu > 60 %, ou p95 > 800 ms
2	Registre comptable en partie double	Premier ecart de solde vendeur non explique
3	Bus de messages	Un module doit vivre a un rythme different
4	Extraction d'un module	Le palier 3 tient depuis 3 mois

1. Le principe qui gouverne tout le document

Le monolithe modulaire n'est pas une etape vers les microservices : c'est une architecture a part entiere, et pour une place de marche a ce stade, c'est la bonne. Un seul processus signifie une seule transaction, un seul deploiement, une seule trace a lire quand quelque chose casse. Ce sont des avantages enormes qu'on ne rend jamais.

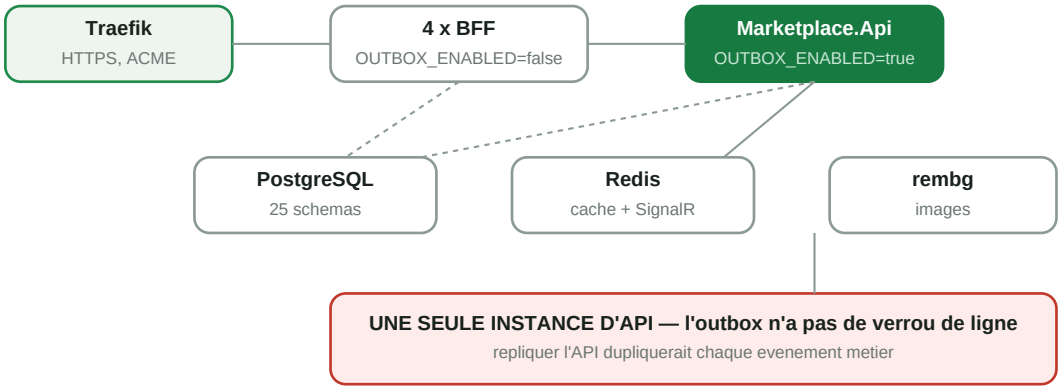
L'erreur classique consiste a decouper par anticipation. On paie alors immediatement la coherence eventuelle, les appels reseau, la duplication des donnees et l'observabilite distribuee — pour un trafic qui ne les justifie pas encore, et parfois jamais. La regle retenue ici est inverse : **on n'extrait un module que lorsqu'une mesure le reclame**, et l'architecture actuelle est deja concue pour que cette extraction reste possible.

Ce qui rend l'extraction future peu couteuse, et qui existe deja

Acquis	Ce qu'il epargnera le jour venu
Un schema PostgreSQL par module	Extraire les donnees d'un module revient a deplacer un schema, pas a demeler des tables.
Aucune cle etrangere entre schemas	Rien a couper : les references sont deja des identifiants nus.
16 interfaces *ModuleApi* dans *Contracts*	La frontiere du service existe deja ; seul son transport changerait.
38 evenements d'integration via outbox	Le contrat asynchrone est ecrit et eprouve ; il suffirait d'en changer le relais.
Regles de dependance testees (NetArchTest)	Le cloisonnement ne s'erode pas entre deux refontes. C'est ce qui garde l'option ouverte.

> **Autrement dit, le travail d'extraction est deja fait a 80 %.** Ce qui reste — le transport reseau, la coherence eventuelle assumee, l'exploitation de N services — est precisement la part qu'il faut retarder le plus longtemps possible.

2. Le point de depart, et sa contrainte



Composition deployee aujourd'hui. En rouge, la contrainte qui commande la suite.

Le code documente lui-meme la limite, dans `OutboxProcessor` : la lecture des messages n'est pas protegee par un `SELECT ... FOR UPDATE SKIP LOCKED`. Deux processeurs concurrents liraient les memes lignes et les publieraient deux fois. C'est pourquoi les quatre BFF posent `OUTBOX_ENABLED=false` et que seule l'API draine.

> **Consequence, ecrite noir sur blanc dans le fichier :** repliquer l'API sans implementer d'abord le verrou de ligne crediterait **chaque gain vendeur autant de fois qu'il y a de repliques**. Ce n'est pas une degradation de performance, c'est une erreur comptable silencieuse. Tant que ce verrou n'existe pas, la seule mise a l'echelle disponible est verticale — une machine plus grosse.

A l'inverse, les BFF Mobile et Vendeur sont deja replicables : leur concentrateur SignalR est adosse au backplane Redis (`signalr.AddStackExchangeRedis`), et ils ne drainent pas l'outbox. C'est une asymetrie utile : la charge de facade peut etre absorbee avant meme le palier 0.

3. Palier 0 — lever le verrou de l'outbox

Signal : aucun. C'est le prealable de tous les autres paliers, et le seul chantier de ce document qu'il faut engager sans attendre de mesure.

Contenu

Passer la lecture de l'outbox en `FOR UPDATE SKIP LOCKED`, de sorte que N processeurs se partagent les messages sans jamais lire les memes. PostgreSQL le fait nativement ; le lot est de l'ordre de la cinquantaine de lignes de code et d'un test d'integration a deux processus concurrents.

```
-- forme cible de la lecture, par module
SELECT * FROM ."OutboxMessages"
 WHERE "ProcessedOnUtc" IS NULL
   AND "DeadLetteredOnUtc" IS NULL
   AND "NextAttemptAtUtc" <= now()
 ORDER BY "OccurredOnUtc"
 LIMIT 50
   FOR UPDATE SKIP LOCKED;
```

Ce que cela debloque

La replication de l'API — donc la tolerance a la panne d'une instance, les deploiements sans coupure, et toute absorption de charge au-dela de ce qu'une seule machine encaisse. Tant que ce palier n'est pas franchi, l'API est un **point de defaillance unique** : son redemarrage est une interruption de service, et aucun palier suivant n'a de sens.

A verifier en meme temps

Les trois autres services de fond — `WithdrawalReconciliationService`, `DiscountExpiryService`, `DependencyHealthPublisher` — subissent la meme question. Deux instances qui reconcilient les memes retraits, ou qui expirent les memes remises, ne produisent pas forcement une erreur, mais personne ne l'a verifie. Le palier 0 est le bon moment pour trancher : verrou de ligne, bail distribue dans Redis, ou drapeau d'activation sur une seule instance.

4. Palier 1 — tenir la charge sans rien extraire

Signal : CPU soutenu au-dela de 60 % sur l'API, ou latence p95 des routes de lecture au-dela de 800 ms, ou une indisponibilite causee par un redemarrage.

Chantier	Pourquoi
API en 2+ repliques	Devient possible apres le palier 0. Supprime le point de defaillance unique et permet le deploiement sans coupure.
Migrations hors du demarrage	Aujourd'hui l'API migre au boot. Avec N repliques, N processus tenteraient de migrer en meme temps. Une etape de deploiement dediee — conteneur ephemere, une seule fois — retire la question. Elle regle aussi la dependance implicite actuelle : un BFF peut demarrer avant que l'API n'ait fini de migrer.
Idempotence persistee	La cle d'idempotence du paiement vit dans Redis. Un vidage du cache rouvre la fenetre du double paiement. Sur un flux qui manipule de l'argent, cette protection appartient a la base.
Lecture separee de l'ecriture	Une replique en lecture PostgreSQL pour le catalogue, la recherche et les tableaux de bord. Ces charges sont massivement lues et tolerent parfaitement quelques secondes de retard.
SLO explicites	OpenTelemetry et Serilog sont deja en place ; ce qui manque est le seuil au-dela duquel quelqu'un est reveille. Sans SLO, les paliers suivants se decident au ressenti.

Ce palier ne change aucune frontiere. C'est ce qui en fait le meilleur rapport effort / effet : il multiplie la capacite sans introduire la moindre coherence eventuelle supplementaire. La plupart des places de marche n'ont jamais besoin d'aller plus loin.

Trois metriques a instrumenter en priorite

Metrique	Ce qu'elle revele	Seuil d'alerte propose
Age du plus vieux message d'outbox non traite	Le relais prend du retard : les modules divergent silencieusement.	> 60 s
Nombre de messages en lettre morte	Perte metier averee — un evenement ne sera jamais applique.	> 0
Paiements en attente depuis plus de 15 minutes	Encaissements Mobile Money suspendus : de l'argent peut avoir ete preleve.	> 0

5. Palier 2 — le registre comptable en partie double

Signal : le premier ecart de solde vendeur que personne ne sait expliquer. Il arrivera : Settlement porte deja 12 migrations, le record de la solution.

C'est le seul palier de ce document qui touche au modele metier, et le seul dont l'absence coute de l'argent reel plutot que des millisecondes. Aujourd'hui, le solde d'un vendeur est un **etat** que plusieurs chemins modifient : encaissement, commission, remboursement, retour, litige, versement. Quand deux d'entre eux se contredisent, il n'existe aucune trace permettant de dire lequel a tort.

Un registre en partie double inverse la charge de la preuve : le solde cesse d'etre stocke, il devient la **somme des ecritures**. Chaque mouvement porte son origine, son montant et sa contrepartie. Un ecart n'est alors plus un mystere mais une ligne qu'on retrouve.

```
Aujourd'hui                          Cible
-----------                          -----
Wallet.Balance  <- etat mute         Ledger : suite d'ecritures immuables
  += encaissement                      (commande #123, +2 650, encaissement)
  -= commission                        (commande #123,   -265, commission 10 %)
  -= remboursement                     (retour  #45,   -1 150, remboursement)
  -= versement                         (lot     #7,    -1 235, versement MoMo)
                                     Solde = SUM(ecritures) — toujours reconstructible
```

Deux benefices immediats, au-dela de l'exactitude

Le vendeur obtient un **releve** qu'il peut verifier lui-meme, ce qui supprime la classe entiere des litiges « je ne comprends pas mon solde ». Et la plateforme obtient une piste d'audit — indispensable le jour ou une autorite, un partenaire bancaire ou un investisseur demandera comment les fonds circulent.

> **C'est aussi la reponse structurelle au probleme FedaPay.** Un encaissement Mobile Money valide tardivement par l'acheteur devient, dans ce modele, une ecriture qui arrive en retard — pas un etat a corriger. La reconciliation periodique cesse d'etre un rattrapage et devient le fonctionnement normal du registre.

6. Palier 3 — le bus de messages

Signal : un module doit vivre a un rythme different des autres — pics de notifications qui ralentissent les commandes, reindexation de recherche qui sature l'API, ou besoin de rejouer des evenements pour reconstruire un modele de lecture.

Il n'y a aujourd'hui **aucun courtier** : l'outbox publie en processus, vers des abonnes qui vivent dans le meme executable. C'est un choix sain — un courtier de plus est un composant de plus a exploiter — et il tient tant que tous les modules partagent le meme rythme.

Le point important est que **ce palier ne change pas le code metier**. Les 38 evenements d'integration existent, leur contrat est stable, l'outbox garantit deja qu'ils sont ecrits dans la meme transaction que le changement. Seul le relais change de nature : au lieu de distribuer en memoire, il publie sur un courtier. Le motif *transactional outbox* a ete concu exactement pour cette bascule.

Ce qui change	Ce qui ne change pas
Le relais : de la distribution en memoire vers une publication sur courtier (RabbitMQ suffit largement).	Les 38 contrats d'evenements, ecrits dans *Contracts*.
Les abonnes deviennent des consommateurs, avec leur propre rythme et leur propre file.	L'ecriture en outbox, dans la transaction metier.
L'exploitation : une file a surveiller, un composant a sauvegarder.	Les regles de dependance et les tests d'architecture.

A ce stade, la coherence eventuelle devient visible pour l'utilisateur : un avis publie peut mettre quelques secondes a apparaitre dans la recherche. C'est acceptable — a condition que l'interface le dise, plutot que de laisser croire a une panne.

7. Palier 4 — extraire un module

Signal : le palier 3 fonctionne depuis au moins trois mois, ET un module precis remplit au moins deux des criteres ci-dessous. Un seul critere ne suffit jamais.

Critere	Formulation concrete
Rythme	Le module doit etre deploye a une cadence propre, ou mis a l'echelle independamment.
Disponibilite	Sa panne ne doit pas emporter le reste — ou l'inverse.
Technologie	Il gagnerait a une pile differente (moteur d'indexation, calcul).
Equipe	Une equipe distincte en a la charge et se marche sur les pieds.
Donnees	Son volume ou son profil d'acces degrade la base commune.

Candidats, par ordre de pertinence

Rang	Module	Pourquoi lui, et ce qu'il faut savoir
1	**Search**	Deja isole (un seul agregat, *Documents*), alimente exclusivement par evenements, aucun appel entrant critique. C'est aussi celui qui gagnerait le plus a une autre technologie. Le candidat naturel — et le moins risque.
2	**Notifications**	Charge en pics, dependance externe (Firebase), aucune donnee dont un autre module depend en lecture. Sa lenteur ne doit jamais ralentir une commande.
3	**Payments + Settlement**	Profil de disponibilite et d'audit different du reste. **Mais a n'extraire qu'APRES le palier 2** : sortir la comptabilite avant d'avoir un registre fiable reviendrait a distribuer le probleme au lieu de le resoudre.
—	**Ordering, Catalog, Identity**	A ne pas extraire. Ce sont les carrefours : presque tous les autres modules les lisent. Les mettre derriere un appel reseau transformerait chaque parcours en chaine de dependances distantes.

> **Un module extrait ne revient jamais.** C'est une decision a sens unique, et elle se paie tous les jours : un deploiement de plus, une trace de plus a correler, une panne partielle de plus a imaginer. Le fait que l'architecture actuelle la rende *possible* n'est pas une raison de la prendre.

8. Ce qu'il ne faut pas faire

Cette section a autant de valeur que les precedentes. Chacune de ces idees paraitra raisonnable a un moment ou a un autre.

1. Un microservice par module

Vingt-cinq services pour une equipe de cette taille, c'est vingt-cinq deploiements, vingt-cinq journaux et une coherence eventuelle partout — pour un trafic qu'une seule machine encaisse encore. Les criteres du palier 4 existent precisement pour empecher ce reflexe.

2. Une base de donnees par module, tout de suite

Le cloisonnement par schema apporte deja l'essentiel du benefice. Separer les bases avant d'avoir un besoin mesure supprime les transactions inter-modules et impose une compensation applicative — beaucoup de complexite pour aucun gain immediat.

3. Kubernetes

Le compose actuel est lisible et suffisant. L'orchestrateur ne se justifie qu'a partir du moment ou plusieurs services ont des cycles de vie reellement independants, c'est-a-dire apres le palier 4.

4. CQRS et event sourcing generalises

Le registre comptable du palier 2 est un cas ou l'evenement EST la verite metier — c'est justifie. Appliquer le meme motif au catalogue ou au panier ajouterait une indirection que rien ne demande.

5. Reecrire les BFF en passerelles HTTP

Ils referencent aujourd'hui le projet API et appellent le metier en processus : zero latence reseau, une seule trace. Les transformer en clients HTTP ajouterait un saut reseau par appel et n'apporterait rien tant que l'API n'est pas extraite.

9. Chantiers transverses, independants des paliers

Chantier	Contenu
Fonctions debranchees	Les zones de livraison (domaine ecrit, aucune persistance, aucun usage) et les regles de commission (modelisees, migrees, editables — sans aucun consommateur). Brancher ou supprimer : laisser en l'etat coute de la maintenance et induit en erreur.
Reconciliation des encaissements	Le pendant entrant de `WithdrawalReconciliationService`. Le palier 2 le rend structurel, mais il est utile bien avant.
Secrets	Rotation des cles PSP et remplissage du coffre. Prealable a toute ouverture d'equipe.
Suppression du code mort	Les consoles Blazor decommissionnees vivent encore dans le depot. Chaque relecture les traverse.

> **Sur le statut de ce document.** Il decrit une trajectoire, pas un engagement. Les signaux declencheurs comptent plus que l'ordre des paliers : si aucun signal ne se declenche, la bonne decision est de ne rien faire au-dela du palier 0 — qui, lui, doit etre engage independamment de toute mesure.